

Dossier complet d'utilisation de tous les produits logiciels

Projet UpcycleConnect Renova Systems

Faty Mutesi · Ndoya Diop · Amel Boudjenane Année 2025-2026

URL publique : <https://upcycleconnect.pro/>

Sommaire

1. Diagnostic du projet
2. Présentation générale
3. Prérequis et accès
4. Comptes de démonstration
5. Guide utilisateur — Visiteur
6. Guide utilisateur — Particulier
7. Guide utilisateur — Professionnel
8. Guide utilisateur — Salarié
9. Guide Administrateur
10. Guide de l'API (Go)
11. Installation locale
12. Base de données
13. Docker, Nginx & déploiement
14. Dépannage rapide
15. Captures d'écran à ajouter
16. Checklist de rendu final

00 — Diagnostic du projet

Analyse factuelle du dépôt `renova-systems-projet-annuel` (produit **UpcycleConnect**, prestataire **Renova Systems**).

Structure du projet

```
renova-systems-projet-annuel/
├── API/                                ← Back-end Go (API REST, port 8081)
│   ├── main.go                        ← point d'entrée (enregistre les routes + ListenAndServe :8081)
│   ├── route/                         ← déclaration des routes HTTP (par domaine)
│   ├── admin/                         ← handlers HTTP (logique des endpoints) + upload, pdf, stripe, webhook
│   ├── bdd/                           ← accès base (une fonction = une requête SQL) + db.go (connexion)
│   ├── models/                       ← structs Go (User, Annonce, Evenement...)
│   ├── auth/jwt.go                   ← JWT + middlewares de rôle
│   └── Dockerfile                    ← build multi-stage du back-end
├── Frontend/                          ← Front-end statique (HTML/CSS/JS) servi par Nginx
│   ├── *.html                        ← pages (login, espClient, espPro, admin_*, salarie/*, annonceAll...)
│   ├── script/                       ← JS (config.js, login.js, dashClient.js, admin/*, salarie/*, notifCloche.js)
│   ├── style/                        ← CSS
│   ├── assets/                       ← logo + favicon (logoUpcycle.png / .ico)
│   ├── nginx.conf                    ← config Nginx (proxy /api, /uploads, URLs propres)
│   └── Dockerfile                    ← build du conteneur front (nginx)
├── db/
│   ├── init.sql                      ← schéma + données (import auto au 1er démarrage Docker)
│   ├── export_db_vide.sql             ← EXPORT structure seule (génééré)
│   ├── export_db_remplie.sql          ← EXPORT structure + données (génééré)
│   └── fix_translations_pro.sql       ← correctif d'accents (utilitaire)
├── docker-compose.yml                ← stack LOCALE (build local)
└── docker-compose-prod.yml           ← stack VM/PROD (images Docker Hub amelbdj/upcycle-*)
```

```
└─ .env.example      ← modèle de variables d'environnement
└─ pa2026.sql        ← dump SQL de référence (historique)
```

Technologies détectées

Élément	Technologie	Preuve
Back-end	Go (net/http , sans framework)	API/main.go , API/go.mod
Base de données	MySQL 8	docker-compose.yml (mysql:8.0), API/bdd/db.go
Front-end	HTML / CSS / JS vanilla + Nginx	Frontend/ , Frontend/nginx.conf
Conteneurs	Docker + Docker Compose	docker-compose.yml , docker-compose-prod.yml , Dockerfiles
Authentification	JWT (HS256)	API/auth/jwt.go
Paieement	Stripe (Checkout + Connect)	API/admin/stripe.go , webhook.go
Notifications push	OneSignal	API/admin/notifications.go
PDF	go-pdf/fpdf (serveur) + jsPDF (client)	API/admin/pdf.go , Frontend/script/admin/box.js

Emplacements clés

Élément	Chemin
Front-end	Frontend/
API Go	API/ (entrée API/main.go)
Base de données (schéma + seed)	db/init.sql
Exports SQL	db/export_db_vide.sql , db/export_db_remplie.sql
Docker (local)	docker-compose.yml
Docker (prod/VM)	docker-compose-prod.yml
Nginx	Frontend/nginx.conf
Variables d'environnement	.env.example (à copier en .env)
Connexion DB (code)	API/bdd/db.go
Config API côté front	Frontend/script/config.js

Fonctionnalités réellement présentes

- Authentification (inscription, connexion, mot de passe oublié) — **JWT**.
- Gestion des utilisateurs et **rôles/permissions** (Utilisateur, Pro, Salarié, Administrateur).
- **Annonces** (dons/ventes) : création, upload photo, validation admin, filtres par catégorie.
- **Catégories** : liste + ajout + suppression (back-office).
- **Conteneurs / box** : réservation → dépôt (PIN) → retrait (code-barres), simulateur matériel.
- **Commandes / commission** (5 %) via Stripe ; suivi finances admin.
- **Upcycling Score** (impact écologique).
- **Événements / formations** (création salarié, inscription, paiement Stripe).
- **Articles / actualités, Forum, Messagerie (WebSocket)**.
- **Espace Pro** : abonnements (freemium/premium), projets avant/après, sponsoring d'annonces.
- **Back-office admin** : validations, utilisateurs (pagination), conteneurs, finances, documents.
- **Notifications** : en base (cloche) + push OneSignal.
- **Génération PDF** : factures/contrats (serveur), rapport logistique (client).
- **Multilingue** (FR/EN) : import de langue depuis l'admin.
- **Déploiement Docker** + Nginx reverse-proxy + HTTPS (URL publique).

Fonctionnalités absentes ou à ne pas promettre

Élément	État
Documentation Swagger/OpenAPI	Non présent dans le code actuel (doc API générée à partir des routes)
Tests automatisés (<code>*_test.go</code>)	Non présent dans le code actuel (tests manuels curl/navigateur)
Stockage de la date de naissance	Non présent (colonne absente ; la vérif d'âge 18+ est côté front)
Vérification réelle du SIRET (API INSEE)	Non présent — seule la clé de Luhn est validée (format)
Calcul du score par matériau	Présent partiellement — le coefficient retombe sur « autre » (le matériau n'est pas relu dans <code>CalculateAndAddScore</code>)

Points à compléter manuellement

- **Mots de passe des comptes de test** : hashés (bcrypt) en base → non extractibles, à renseigner (voir `03_COMPTES_DE_DEMONSTRATION.md`).
- **Captures d'écran** du dossier (voir `14_CAPTURE_ECRANS_A_AJOUTER.md`).
- Secrets (Stripe/OneSignal/JWT) : valeurs de repli présentes dans le code → à externaliser en `.env` pour la prod.

01 — Présentation générale

UpcycleConnect en une phrase

UpcycleConnect est une plateforme web d'**économie circulaire** qui met en relation des particuliers, des professionnels (artisans) et l'équipe interne pour **donner, vendre, récupérer et transformer** des objets et matériaux, via un réseau de **conteneurs connectés**.

- **Produit** : UpcycleConnect
- **Prestataire** : Renova Systems
- **URL publique** : <https://upcycleconnect.pro/>

Objectif métier

Réduire le gaspillage en donnant une seconde vie aux objets et matériaux : - un **particulier** dépose un objet (don ou vente) dans un casier d'un conteneur ; - un **professionnel/artisan** récupère la matière pour la transformer (projets d'upcycling) ; - la plateforme **mesure l'impact** (Upcycling Score, CO₂ évité) et **sécurise les transactions** (paiement Stripe, commission).

Public cible

- **Particuliers** soucieux de donner/vendre plutôt que jeter.
- **Professionnels / artisans** en quête de matière première de réemploi.
- **Collectivités / structures** exploitant le réseau de conteneurs (via l'équipe interne).

Rôles utilisateurs

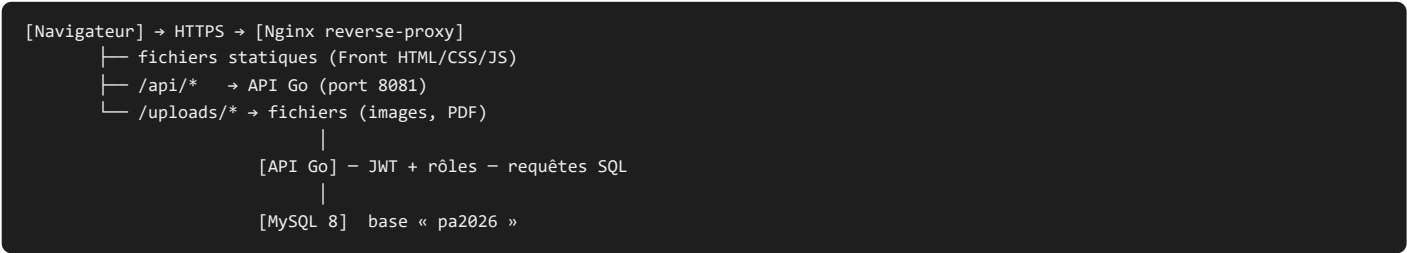
Rôle	Description	Espace
Visiteur	Non connecté : accès aux pages publiques (accueil, connexion, inscription)	—
Utilisateur (Particulier)	Crée des annonces, dépose/récupère en conteneur, suit son score éco	<code>/client</code>
Professionnel (Pro)	Consulte/achète les annonces, gère abonnements et projets d'upcycling	<code>/pro</code>
Salarié	Publie événements/formations et articles (soumis à validation)	<code>/salarie</code>
Administrateur	Back-office complet : utilisateurs, validations, conteneurs, finances, documents	<code>/admin</code>

Fonctionnalités principales

- Comptes et **authentification sécurisée** (JWT) avec 4 rôles.
- **Annonces** de dons/ventes avec photos, catégories et **validation** par l'admin.
- **Réseau de conteneurs** : réservation d'un casier, dépôt par code PIN, retrait par code-barres.
- **Paiement en ligne** (Stripe) avec **commission de 5 %** reversée à la plateforme.

- **Upcycling Score** : mesure de l'impact écologique de chaque utilisateur.
- **Événements & formations** organisés par les salariés.
- **Espace professionnel** : abonnements (freemium/premium), projets avant/après, sponsoring.
- **Notifications** (cloche in-app + push), **multilingue** (FR/EN), **factures PDF**.
- **Back-office** d'administration complet.

Architecture logicielle (résumé)



- **Front** et **API** sont **séparés** et déployés dans des conteneurs Docker distincts.
- Toute action sensible est **vérifiée côté serveur** (le front n'est jamais une source de confiance).
- Les fichiers uploadés sont stockés dans un **volume Docker persistant** (jamais en base).

02 — Prérequis et accès

Accès simple (utilisateur / correcteur qui veut juste tester en ligne)

- **Aucune installation.** Un navigateur récent suffit.
- **Navigateur recommandé** : Google Chrome ou Microsoft Edge à jour (Firefox fonctionne également).
- **URL publique** : <https://upcycleconnect.pro/>
- Se connecter avec un **compte de démonstration** (voir 03_COMPTES_DE_DEMONSTRATION.md).

Accéder au site

1. Ouvrir le navigateur.
2. Saisir l'adresse **https://upcycleconnect.pro/** → la page d'accueil s'affiche.
3. Cliquer sur **Connexion** (URL `/login`) et saisir un compte de démonstration.
4. Selon le rôle, l'utilisateur est redirigé vers son espace (`/client` , `/pro` , `/salarie` , `/admin`).

Prérequis développeur / correcteur (pour lancer en local)

Le projet est **entièrement conteneurisé** : le seul prérequis obligatoire est **Docker**.

Logiciel	Nécessaire ?	Usage
Docker Desktop + Docker Compose	✔ Obligatoire	Lance toute la stack (base, API, front)
Git	Recommandé	Récupérer le dépôt (<code>git clone</code>)
Go (1.25)	Optionnel	Uniquement pour lancer l'API sans Docker
MySQL 8	Optionnel	Uniquement en mode « sans Docker » (sinon fourni par le conteneur)
Node / npm	✗ Non requis	Le front est du HTML/JS statique, pas de build
PostgreSQL	✗ Non	Le projet utilise MySQL , pas PostgreSQL

Résumé : avec **Docker Desktop** seul, tout fonctionne. Go/MySQL ne servent que pour un lancement manuel avancé.

URL du projet

Élément	URL
Site public (prod)	https://upcycleconnect.pro/
API en prod	https://upcycleconnect.pro/api/... (proxy Nginx → back-end)
Front en local (Docker)	http://localhost:8088/ (port configurable via WEB_PORT)
API en local (Docker)	http://localhost:8081/... (exposée pour debug)

Détail de la communication front ↔ API

Le fichier `Frontend/script/config.js` choisit automatiquement l'adresse de l'API : - **Développement WAMP local** (URL contenant `/Frontend/`) → `http://localhost:8081` . - **Docker / prod** → `origin + "/api"` (ex : `https://upcycleconnect.pro/api`), le proxy Nginx retirant le préfixe `/api` .

03 — Comptes de démonstration

Ces comptes existent réellement dans la base fournie (`db/export_db_remplie.sql` , table `utilisateur`). **Les mots de passe sont hashés (bcrypt) en base** : ils ne peuvent pas être extraits du dump. → **à compléter** avec le mot de passe défini lors du seed.

Comptes disponibles (validés)

Rôle	Email	Mot de passe	Utilité du compte	Parcours à tester
Administrateur	<code>test.admin@renova.test</code>	<i>à compléter</i>	Back-office complet	Valider un compte / une annonce, voir finances, conteneurs, catégories
Administrateur	<code>test.admin@test.fr</code>	<i>à compléter</i>	(secours)	idem
Salarié	<code>test.salarie@renova.test</code>	<i>à compléter</i>	Espace interne	Créer un événement/formation, un article (soumis à validation)
Salarié	<code>test.salarie@test.fr</code>	<i>à compléter</i>	(secours)	idem
Professionnel	<code>test.pro@renova.test</code>	<i>à compléter</i>	Espace pro	Consulter les annonces, abonnement, projet upcycling, sponsoring
Professionnel	<code>test.pro@test.fr</code>	<i>à compléter</i>	(secours)	idem
Particulier	<code>test.client@renova.test</code>	<i>à compléter</i>	Espace client	Créer une annonce, suivre son statut, score éco, conteneurs
Particulier	<code>test.user@test.fr</code>	<i>à compléter</i>	(secours)	idem

Vérifier la liste à jour : `SELECT email, role, validation FROM utilisateur WHERE email LIKE 'test.%';`

Visiteur (sans compte)

Aucun identifiant : accès aux pages publiques uniquement (accueil, connexion, inscription). Voir `04_GUIDE_UTILISATEUR_VISITEUR.md` .

Si vous devez (re)créer des comptes de démonstration

1. **Par l'inscription** : page `/register` (rôles Particulier et Professionnel uniquement — l'inscription en Admin/Salarié est volontairement bloquée côté serveur pour la sécurité).
2. **Par l'administration** : `/admin` → carte/section Utilisateurs → « Ajouter » (permet de choisir n'importe quel rôle, y compris Salarié/Admin).
3. Un compte fraîchement inscrit peut être « **En attente** » → le valider depuis l'admin avant la démonstration.

Liste de comptes recommandés à préparer pour une démo

Rôle	Email suggéré	Mot de passe suggéré
Administrateur	<code>test.admin@renova.test</code>	(à définir, ex : <i>Admin2026!</i>)
Salarié	<code>test.salarie@renova.test</code>	(à définir)

Rôle	Email suggéré	Mot de passe suggéré
Professionnel	test.pro@renova.test	(à définir)
Particulier	test.client@renova.test	(à définir)

Pour redéfinir un mot de passe connu sur un compte de test, le plus simple est de recréer le compte via `/register` (particulier/pro) ou via l'admin (tous rôles), puis de le valider.

04 — Guide utilisateur : Visiteur (non connecté)

Le **visiteur** est un internaute non authentifié. Il a accès aux pages publiques mais pas aux espaces personnels.

Accéder à la page d'accueil

- Ouvrir **https://upcycleconnect.pro/** (URL `/`).
- La page d'accueil (landing) présente le concept UpcycleConnect.
- Le **sélecteur de langue** (FR/EN) est disponible en bas à droite de l'écran.

(Capture à ajouter : `accueil.png`.)

Pages accessibles sans compte

Page	URL propre	Contenu
Accueil	<code>/</code>	Présentation du service
Connexion	<code>/login</code>	Formulaire d'authentification
Inscription	<code>/register</code>	Création de compte (Particulier ou Professionnel)
Mot de passe oublié	<code>/reset-password</code>	Réinitialisation
À propos	<code>/a-propos</code>	Informations sur le projet

Les pages « métier » (annonces, espaces client/pro/salarié/admin) exigent une **connexion**. Un garde de session (`Frontend/script/auth_guard.js`) redirige vers `/login` si aucun jeton n'est présent.

Consulter / créer un compte

S'inscrire

- Cliquer sur **S'inscrire** (`/register`).
- Choisir un **profil** : **Particulier** ou **Professionnel** (les rôles Salarié/Administrateur ne sont pas proposés à l'inscription).
- Remplir le formulaire :
- Particulier** : nom, prénom, email, mot de passe, **date de naissance** (l'inscription est refusée si moins de **18 ans**).
- Professionnel** : + **nom d'entreprise** et **SIRET** (14 chiffres, validé par clé de contrôle).
- Valider → le compte est créé. Un **Particulier** est actif immédiatement ; un **Professionnel** peut être « **En attente** » de validation par un administrateur.

Se connecter

- Cliquer sur **Connexion** (`/login`).
- Saisir email + mot de passe → redirection automatique vers l'espace correspondant au rôle.

(Captures à ajouter : `connexion.png`, `inscription.png`.)

Limitations d'un visiteur

- Ne peut **pas** créer d'annonce, ni déposer/récupérer en conteneur.

- Ne voit **pas** les espaces client/pro/salarié/admin.
- Doit **s'inscrire ou se connecter** pour toute action personnalisée.

05 — Guide utilisateur : Particulier

Le **Particulier** donne ou vend des objets et suit son impact écologique. Espace : `/client`.

1. Connexion

1. Ouvrir `/login`.
2. Saisir l'email et le mot de passe du compte particulier (voir `03_COMPTES_DE_DEMONSTRATION.md`).
3. **Résultat** : redirection automatique vers `/client` (Espace Particulier).

(Capture : `connexion.png`.)

2. Tableau de bord

- **Page** : `/client`.
- Affiche en haut trois indicateurs **dynamiques** : **Annonces** (nombre d'annonces actives), **Dépôt actif** (objets en conteneur), **Score éco**.
- Plus bas : la liste **Mes Annonces** et le **Système de Conteneurs**.

(Capture : `dashboard_particulier.png`.)

3. Créer une annonce

1. Sur `/client`, cliquer sur **+** (**Ajouter une annonce**) dans la section « Mes Annonces ».
2. Remplir le formulaire :
3. **Titre**, **Catégorie** (liste issue de la base : Textile, Bois, Plastique, Métal), **Type** (Vente / Don gratuit), **Prix** (si vente), **Description**.
4. **Photo de l'objet** : bouton **Choisir un fichier** (formats image, taille max 5 Mo).
5. Cliquer sur **Soumettre l'annonce**.
6. **Résultat attendu** : message « Votre annonce sera soumise à validation avant d'être publiée (délai 24 h max) ». L'annonce apparaît avec le statut **En attente**.

(Captures : `creation_annonce.png`, `upload_photo.png`.)

4. Suivre le statut d'une annonce

- Dans « Mes Annonces », chaque carte affiche un **badge de statut** :
- **En attente** : en cours de validation par l'admin.
- **Validée** : publiée et visible sur la marketplace.
- **Rejetée** : refusée par l'admin.
- Boutons disponibles : **Modifier** et **Supprimer**.

5. Consulter / rechercher les annonces

- **Page** : `/annonces`.
- **Filtres** : par **catégorie** (dynamique depuis la base) et par **type** (Vente / Don).
- **Recherche** par mot-clé, **tri** (plus récentes, prix, A→Z), vue grille/liste.

(Capture : `marketPlace.png`.)

6. Conteneurs : dépôt et retrait

Quand un objet est vendu/réservé, un **casier** est attribué : 1. Le particulier obtient un **code PIN** (dépôt) et un **code-barres** (retrait), visibles dans la section conteneurs de `/client`. 2. **Dépôt** : au conteneur, saisir le **PIN** → l'objet est marqué déposé. 3. **Retrait** : l'acheteur scanne le **code-barres** → l'objet est récupéré.

- Un **simulateur** (`/simulateur`) permet de tester dépôt et retrait sans matériel physique.

(Capture : `conteneurs_particulier.png`.)

7. Upcycling Score

- **Page** : `/client` , encart « Mon Upcycling Score ».
- Le score augmente lorsqu'un objet déposé est **récupéré** (calcul : poids × coefficient).
- Affiche aussi les statistiques d'impact (objets donnés, déchets évités).

8. Notifications

- Une **cloche** 🛎 affiche une pastille rouge en cas de notification non lue.
- **Clic** sur la cloche → panneau listant les notifications (ex : annonce validée). Les notifications passent en « lues » à l'ouverture.

9. Profil

- **Page** : `/profil` (accessible via l'avatar en haut à droite). Permet de consulter/mettre à jour ses informations. La couleur de la page s'adapte au rôle.

06 — Guide utilisateur : Professionnel / Artisan

Le **Professionnel** récupère de la matière première de réemploi et gère ses projets d'upcycling. Espace : `/pro` (interface en teal).

1. Connexion

1. Ouvrir `/login` , saisir un compte **Pro** (voir `03_COMPTES_DE_DEMONSTRATION.md`).
2. **Résultat** : redirection vers `/pro` (Espace Professionnels & Artisans).

Un compte Pro peut être « **En attente** » de validation par un administrateur après inscription (une pièce justificative / SIRET est demandée).

2. Tableau de bord Pro

- **Page** : `/pro` .
- Indicateurs : **Projets actifs**, **CO₂ évité**.
- Encart **abonnement** (Freemium / Premium / Pro).

(Capture : `dashboard_pro.png` .)

3. Catalogue / consultation des annonces

- **Page** : `/annonces` (mêmes annonces que les particuliers, mais l'interface prend les **couleurs pro**).
- Filtres par catégorie/type, recherche, tri.
- **Réservation / achat** : le paiement passe par **Stripe** ; une **commission de 5 %** est prélevée par la plateforme.

4. Abonnements (Freemium / Premium)

- **Freemium** (gratuit) : dépôt de matériaux, accès aux annonces de base.
- **Premium / Plus / Pro** (payant) : tableaux de bord avancés, alertes prioritaires, visibilité accrue, factures PDF automatiques.
- **Actions** : « Passer au Premium », gestion via le **portail Stripe**, résiliation.
- Endpoints associés : `POST /api/pro/subscribe` , `/upgrade` , `/cancel` , `/portal` .

(Capture : `abonnement_pro.png` .)

5. Projets d'upcycling

- Depuis `/pro` , section **Projets** : créer un projet avec **photos avant / après** et **CO₂ évité estimé**.
- Suivi par **étapes** (à faire / en cours / terminé).
- Endpoints : `POST /api/pro/projets/create` , `GET /api/pro/projets` , `POST /api/pro/etapes/create` , `PUT /api/pro/etapes/statut` .

(Capture : `projet_pro.png`.)

6. Sponsoring d'annonces

- Un pro peut **sponsoriser** une de ses annonces pour la mettre en avant sur la marketplace.
- Endpoint : `POST /api/pro/annonces/sponsor`.

7. Factures / transactions

- Les paiements (abonnement, achats) génèrent des **factures/contrats PDF** côté serveur.
- Consultables via l'espace pro (`GET /api/pro/invoices`).

8. Limitations selon le rôle

- Un Pro **ne peut pas** accéder au back-office admin ni aux espaces salarié.
- Certaines fonctionnalités (tableaux avancés, alertes prioritaires) sont réservées aux **abonnés Premium**.

07 — Guide utilisateur : Salarié (équipe interne)

Le **Salarié** anime la plateforme : il publie des événements/formations et des articles, soumis à validation de l'administrateur. Espace : `/salarie`.

1. Connexion

- Ouvrir `/login`, saisir un compte **Salarié** (voir `03_COMPTE_DE_DEMONSTRATION.md`).
- Résultat** : redirection vers `/salarie` (Espace Salariés).

2. Espace salarié

- Page d'accueil** : `/salarie` (tableau de bord).
- Sous-espaces :
- Événements** : `/salarie/evenements`
- Planning** : `/salarie/planning`
- Contenus / Articles** : `/salarie/content`
- Forum** : `/salarie/forum`

(Capture : `dashboard_salarie.png`.)

3. Créer un événement / une formation

- Aller sur `/salarie/evenements`.
- Cliquer sur **créer un événement**.
- Remplir : **titre**, **type**, **description**, **date de début/fin**, **lieu**, **capacité**, **tarif**, **image**, éventuellement un **plan (PDF)** et des **ressources pédagogiques (PDF)**.
- Valider.
- Résultat** : « Événement soumis avec succès. Il est en attente de validation. » → statut **En attente**.

Prérequis important : pour déposer un événement **payant**, le salarié doit disposer d'un **compte Stripe** (identifiant `stripe_account_id`). Sans cela, un message invite à créer son compte Stripe.

(Capture : `creation_evenement.png`.)

4. Planning

- Page** : `/salarie/planning` : vue calendrier des événements, gestion des inscrits, des ressources.

5. Articles / actualités

- **Page** : `/salarie/content` : rédaction d'articles (titre, contenu, image). Soumis à **validation admin** avant publication.

6. Forum

- **Page** : `/salarie/forum` : participation aux discussions d'entraide.

7. Actions nécessitant une validation admin

Action du salarié	Validé par
Publication d'un événement / formation	Administrateur (<code>/admin/validations</code>)
Publication d'un article	Administrateur

- Après décision de l'admin (validation ou refus), le salarié reçoit une **notification** (cloche 🔔) l'informant du résultat.

08 — Guide Administrateur (back-office)

L'**Administrateur** dispose d'un back-office complet. Espace : `/admin`. Toutes les routes `/admin/*` sont protégées côté serveur (JWT + rôle).

Connexion

1. Ouvrir `/login`, saisir un compte **Administrateur** (voir `03_COMPTES_DE_DEMONSTRATION.md`).
2. **Résultat** : redirection vers `/admin` (Vue d'ensemble). Le menu latéral donne accès aux modules.

(Capture : `backoffice_admin.png`.)

Module 1 — Vue d'ensemble (`/admin`)

- **Objectif** : synthèse en temps réel.
- **Contenu** : KPI (utilisateurs actifs, revenus du mois, conteneurs/casiers, déchets sauvés), **Centre d'actions requises** (annonces/événements/articles en attente, casiers en panne), **Activité récente** (dernières annonces), **badge de validations** dans le menu.
- **Modules intégrés à cette page** : **Envoyer une notification**, **Catégories** (liste/ajout/suppression), **Langues / Traductions** (import d'une langue).
- **Précaution** : les compteurs sont calculés à chaque chargement ; rafraîchir la page après une action.

Module 2 — Utilisateurs (`/admin/users`)

- **Objectif** : gérer les comptes.
- **Actions** :
 - Lister les utilisateurs (**pagination 10/page**), rechercher, filtrer par rôle.
 - **Valider / Refuser** un compte en attente (bouton ✓ / ✕).
 - **Bannir** un utilisateur.
 - **Ajouter** un utilisateur (choix du rôle, y compris Salarié/Admin).
 - **Modifier / Supprimer** un utilisateur.
- **Résultat attendu** : le statut se met à jour immédiatement dans le tableau.
- **Précaution** : un refus/bannissement peut déclencher un e-mail/une notification ; vérifier l'identité avant d'agir.

(Captures : `validation_utilisateur.png`, `Liste_utilisateurs.png`.)

Module 3 — Validations (`/admin/validations`)

- **Objectif** : modérer les contenus soumis.
- **Onglets** : Tout voir / Annonces / Événements / Contenus (articles).
- **Actions** : **Approuver** ou **Refuser** une annonce, un événement, un article.

- **Résultat attendu** : l'élément validé/refusé **disparaît de la liste en direct** ; le badge de validations se met à jour ; l'auteur reçoit une notification.

(Capture : `validation_annonce.png`.)

Module 4 — Logistique & Box (`/admin/conteneurs`)

- **Objectif** : gérer le réseau physique.
- **Actions** :
 - Voir les conteneurs (nom, adresse, nombre de casiers) et l'état des casiers (libre / occupé / maintenance).
 - **Créer un conteneur, ajouter un casier, changer le statut** d'un casier.
 - **Générer un rapport logistique PDF** (bouton «  Rapport logistique »).
 - **Précaution** : ne pas supprimer un conteneur contenant des objets déposés.

(Capture : `conteneurs.png`.)

Module 5 — Finances (`/admin/finances`)

- **Objectif** : suivre l'activité financière.
- **Contenu** : volume d'affaires du mois, **revenus (commission 5 %)**, tableau des transactions (facture/annonce, montant, commission, statut).
- Endpoints : GET `/admin/finance/overview` , GET `/admin/finance/transactions` .

(Capture : `finances.png`.)

Module 6 — Documents (`/admin/documents`)

- **Objectif** : consulter les factures/contrats PDF.
- **Actions** : lister les documents (type, utilisateur, commande) et **ouvrir le PDF** (« Ouvrir le PDF »).
- Les PDF sont générés côté serveur lors d'un paiement (facture) ou d'une souscription (contrat) et stockés dans le volume `uploads` .

Module 7 — Envoyer une notification (sur `/admin`)

- **Objectif** : communiquer avec une audience.
- **Actions** : choisir la cible (Particuliers / Professionnels / Tous), saisir un message, **Envoyer**.
- **Résultat** : chaque destinataire reçoit la notification (cloche + push OneSignal en prod).

Module 8 — Catégories (sur `/admin`)

- **Objectif** : gérer les catégories de matériaux (Textile, Bois, Plastique, Métal...).
- **Actions** : voir la liste, + **Ajouter** (libellé), **supprimer**.
- **Résultat** : la nouvelle catégorie apparaît immédiatement dans le filtre de la page annonces.

Module 9 — Langues / Traductions (sur `/admin`)

- **Objectif** : ajouter une langue à l'interface (multilingue).
- **Actions** :
 - ↓ **Télécharger le modèle (FR)** → fichier JSON des textes en français.
 - Traduire les **valeurs** (garder les clés) dans le fichier.
 - + **Ajouter une nouvelle langue** → saisir le **code** (ex : `es`), le **nom** (ex : `Español`), sélectionner le fichier, **Importer**.
- **Résultat** : la langue apparaît dans le sélecteur ; l'import est réservé aux administrateurs (route protégée).

Statistiques

- Présentes sous forme de **KPI** sur la Vue d'ensemble et la page Finances (pas de module « statistiques » séparé).

Multilingue

- Géré via `data-i18n` + table `translations` ; import de langue décrit ci-dessus (Module 9).
-

09 — Guide de l'API (Go)

Présentation

L'API est écrite en **Go** avec la bibliothèque standard `net/http` (pas de framework). Elle expose une API REST/JSON, protégée par **JWT** et vérification de **rôle côté serveur**.

- **Emplacement du code** : dossier `API/` (point d'entrée `API/main.go`).
- **Port d'écoute** : **8081**.
- **Organisation** : `route/` (déclare les URLs) → `admin/` (handlers) → `bdd/` (SQL) → `models/` (structs).
- **Swagger/OpenAPI** : **non présent dans le code actuel** — la documentation ci-dessous est générée à partir des routes réelles (`API/route/*.go`).

Lancer l'API

```
# Via Docker (recommandé) — depuis la racine du projet
docker compose up -d --build backend

# Sans Docker (Go + MySQL installés localement)
cd API
go run .           # écoute sur :8081
```

Authentification

- Se connecter via `POST /admin/login` → renvoie un **token JWT**.
- Ajouter ce token sur chaque appel protégé : header `Authorization: Bearer <token>` .
- Le middleware `API/auth/jwt.go` vérifie le token (`VerifyTokenMiddleware`) et le rôle (`VerifyRoleMiddleware`).

```
# Obtenir un token
curl -X POST http://localhost:8081/admin/login \
-H "Content-Type: application/json" \
-d '{"email":"test.admin@renova.test","mot_de_passe":"MOT_DE_PASSE"}'
```

Codes d'erreur possibles

Code	Signification
200 / 201	Succès
400	Requête invalide (données manquantes/malformées)
401	Non authentifié (token absent/expiré/invalide)
403	Rôle non autorisé
404	Ressource/route introuvable
500	Erreur serveur (souvent SQL)

Principales routes (générées depuis le code)

Authentification — `API/route/auth.go` , `users.go`

Méthode	Route	Accès	Description
POST	<code>/admin/login</code>	public	Connexion → token JWT
POST	<code>/auth/inscription</code>	public	Inscription (Particulier/Pro ; Admin/Salarié refusés)
POST	<code>/auth/check-email</code>	public	Vérifier disponibilité email
POST	<code>/auth/forgot-password</code>	public	Demande de réinitialisation
POST	<code>/auth/reset-password</code>	public	Réinitialisation via token

Utilisateurs — API/route/users.go

Méthode	Route	Accès
GET	/admin/users	Salarié/Admin
GET	/admin/users/{id}	token
GET	/admin/users/search?search=	token
GET	/admin/users/role/{role}	token
POST	/admin/users/add	token
PUT	/admin/users/modify/{id}	token
DELETE	/admin/users/delete/{id}	token
PUT	/admin/users/validate refuse ban/{id}	token

Annonces — API/route/annonces.go

Méthode	Route	Accès
GET	/api/annonces/all?id=	public
GET	/mes-annonces?id=	token
GET	/admin/annonces	token
POST	/admin/annonces/add	token (multipart : image)
PUT	/admin/annonces/modify/{id}	token
PUT	/admin/annonces/validate refuse/{id}	token
DELETE	/admin/annonces/delete/{id}	token
POST	/api/payment-annonce	token

Catégories — API/route/categories.go

Méthode	Route
GET	/admin/categories
POST	/admin/categories/add
DELETE	/admin/categories/delete/{id}

Conteneurs / box — API/route/logistique.go

Méthode	Route
GET	/api/admin/conteneurs
GET	/api/admin/conteneur/{id}/boxes
POST	/api/admin/conteneur/create
POST	/api/admin/box/add
PUT	/api/admin/box/update
POST	/api/box/reserve /deposit /collect
POST	/api/hardware/simulate-deposit /simulate-withdrawal
GET	/api/user/boxes?user_id= , /api/user/pickups/{id}

Événements — API/route/evenements.go

Méthode	Route
GET	/admin/evenements

Méthode	Route
POST	/admin/evenements/add (multipart)
PUT	/admin/evenements/validate refuse/{id}
POST	/admin/evenements/inscription desinscription
POST	/api/web/checkout/evenement (Stripe)

Finance — API/route/finance.go

Méthode	Route
GET	/admin/finance/overview
GET	/admin/finance/transactions

Pro / abonnements — API/route/pro.go

POST /api/pro/subscribe|upgrade|cancel|portal · GET /api/pro/sync|invoices|projets|etapes · POST /api/pro/projets/create · POST /api/pro/annonces/sponsor

Forum / Chat — API/route/forum.go , chat.go

GET/POST /user/forums · GET/POST /user/forums/messages · GET /ws/chat (WebSocket) · GET /api/chat/conversations|history

Notifications — API/route/notifications.go

POST /admin/notifications/send · GET /admin/notifications/user/{id} · POST /admin/notifications/user/{id}/read

Traductions — API/route/traductions.go

GET /api/translations?lang=fr · GET /api/languages · POST /admin/translations/add (Admin) · GET /admin/translations/keys

Documents / uploads — API/route/documents.go , auth.go

GET /admin/documents · GET /uploads/* (images, PDF servis par le back-end)

Exemples curl

```
# Lister les utilisateurs (admin)
curl http://localhost:8081/admin/users -H "Authorization: Bearer $TOKEN"

# Lister les annonces publiques
curl "http://localhost:8081/api/annonces/all?id=1"

# Statistiques éco d'un utilisateur
curl "http://localhost:8081/api/user/stats?user_id=1"

# Créer une catégorie
curl -X POST http://localhost:8081/admin/categories/add \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"libelle":"Verre"}'
```

Postman : créer une variable `token` (récupérée via `/admin/login`) et l'utiliser dans l'en-tête `Authorization: Bearer {{token}}` pour tester les routes protégées.

10 — Installation et lancement en local

Méthode recommandée : **Docker Compose** (aucune installation de Go/MySQL requise). Une méthode manuelle est fournie en fin de fiche.

A. Lancement avec Docker Compose (recommandé)

1. Récupérer le projet

```
git clone <URL_DU_DEPOT> upcycleconnect
cd upcycleconnect
```

(ou copier le dossier du projet — celui qui contient `docker-compose.yml`, `API/`, `Frontend/`, `db/`).

2. Configurer les variables d'environnement (optionnel)

Les valeurs par défaut fonctionnent sans `.env`. Pour personnaliser :

```
cp .env.example .env
# éditer .env : mots de passe, WEB_PORT, clés Stripe/OneSignal...
```

Contenu de `.env.example` :

```
WEB_PORT=80
MYSQL_ROOT_PASSWORD=change_me_root
DB_NAME=pa2026
DB_USER=upcycle
DB_PASS=change_me_app
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
ONESIGNAL_APP_ID=
ONESIGNAL_API_KEY=
```

3. Lancer toute la stack

```
WEB_PORT=8088 docker compose up -d --build
```

- Première exécution : build ~2–5 min (téléchargement + compilation Go).
- `WEB_PORT=8088` évite un conflit avec le port 80.
- La **base est importée automatiquement** au premier démarrage via `db/init.sql` — rien à importer manuellement.

4. Vérifier que tout fonctionne

```
docker ps      # 3 conteneurs : uc_mysql (healthy), uc_backend, uc_frontend, (+ uc_phpmyadmin en local)
```

- **Front** : ouvrir `http://localhost:8088/` → la page d'accueil s'affiche.
- **API** : `curl -s -o /dev/null -w "%{http_code}" http://localhost:8081/admin/users` → `401` (API vivante, protégée).
- Se connecter avec un compte de démonstration.

5. Arrêter / relancer

```
docker compose stop  # arrêt (données conservées)
docker compose start  # relance
docker compose down   # supprime les conteneurs (volumes/données conservés)
docker compose down -v # ⚠️ supprime AUSSI les données (réimport de init.sql au prochain up)
```

B. Importer un export SQL manuellement (si besoin)

```
# Base remplie (schéma + données)
docker exec -i uc_mysql mysql --default-character-set=utf8mb4 -uupcycle -pupcyclePass123 pa2026 < db/export_db_remplie.sql
```

```
# Base vide (structure seule)
docker exec -i uc_mysql mysql --default-character-set=utf8mb4 -upcycle -pupcyclePass123 pa2026 < db/export_db_vide.sql
```

C. Lancement manuel (sans Docker) — avancé

Prérequis : **Go 1.25** + **MySQL 8** installés.

```
# 1) Base : créer la base pa2026 et importer un export
mysql -uroot -p -e "CREATE DATABASE pa2026 CHARACTER SET utf8mb4;"
mysql -uroot -p pa2026 < db/export_db_remplie.sql

# 2) API Go (variables d'env par défaut : localhost/root/root/pa2026)
cd API
go build ./...      # vérifie la compilation
go run .           # écoute sur :8081

# 3) Front : servir les fichiers statiques du dossier Frontend
# (via WAMP, ou un serveur statique). Ouvrir la page login.html.
```

En mode WAMP, Frontend/script/config.js détecte l'URL /Frontend/ et pointe l'API sur http://localhost:8081 .

Récapitulatif express

1. Docker Desktop lancé
2. Dossier du projet ouvert dans un terminal
3. WEB_PORT=8088 docker compose up -d --build
4. Attendre ~3 min → http://localhost:8088
5. Se connecter avec un compte de démonstration

11 — Base de données

Informations générales

- **Nom de la base** : pa2026
- **Type** : **MySQL 8** (moteur InnoDB, charset utf8mb4)
- **Utilisateur applicatif** : upcycle (mot de passe local par défaut : upcyclePass123) — root disponible pour l'admin.
- **Connexion depuis l'API** : API/bdd/db.go (NewDB()), identifiants via variables d'environnement.

Emplacement des exports SQL

Fichier	Contenu	Chemin
export_db_vide.sql	Structure seule (CREATE TABLE, sans données)	db/export_db_vide.sql
export_db_remplie.sql	Structure + données (31 tables + jeux de données de démonstration)	db/export_db_remplie.sql
init.sql	Schéma + données importé automatiquement au 1er démarrage Docker	db/init.sql

Les deux fichiers export_db_vide.sql et export_db_remplie.sql sont **fournis** dans db/ . Ils ont été générés avec mysqldump (voir commandes ci-dessous).

Base vide vs base remplie

- **Base vide** (export_db_vide.sql) : uniquement la **structure** (tables, colonnes, clés). Utile pour repartir d'une base neuve et y injecter ses propres données.

- **Base remplie** (`export_db_remplie.sql`) : la structure + des **données de démonstration** (comptes de test, catégories, conteneurs, annonces, événements...). C'est celle à utiliser pour une **démonstration immédiate**.

Commandes d'import

```
# Vers le conteneur MySQL (recommandé) – toujours en utf8mb4 pour les accents
docker exec -i uc_mysql mysql --default-character-set=utf8mb4 -uupcycle -pupcyclePass123 pa2026 < db/export_db_remplie.sql

# Vers un MySQL local (sans Docker)
mysql --default-character-set=utf8mb4 -uroot -p pa2026 < db/export_db_remplie.sql
```

Commandes d'export (régénérer les fichiers)

```
# Base remplie (structure + données)
docker exec uc_mysql mysqldump --default-character-set=utf8mb4 -uupcycle -pupcyclePass123 --databases pa2026 > db/export_db_remplie.sql

# Base vide (structure seule)
docker exec uc_mysql mysqldump --default-character-set=utf8mb4 -uupcycle -pupcyclePass123 --no-data --databases pa2026 > db/export_db_vide.sql
```

Tables principales (31 au total)

Table	Rôle
utilisateur	comptes (rôle, validation, score, siret, stripe_account_id...)
annonce	dons/ventes (titre, prix, type, statut_validation, statut_vente, image, id_categorie, id_user...)
categorie	catégories de matériaux (Textile, Bois, Plastique, Métal)
conteneur , box , box_conteneur	réseau de casiers
depot_box , historique_conteneurs	cycle de vie dépôt/retrait (PIN, code-barres, dates)
order , paiement	commandes et paiements (commission)
document , documents_legaux	factures/contrats PDF, pièces justificatives
evenement , inscription , ressource_pedagogique	événements/formations
article_news	articles/actualités
topic_forum , message_forum , message	forum et messagerie
notification	notifications in-app
abonnement , plan_abo , projet_pro , etapes_projet	espace pro
translations , languages	multilingue
upcycling_score , log_connexion	impact éco, logs

Relations importantes (clés étrangères)

```
utilisateur (1) —< annonce (id_user)
categorie (1) —< annonce (id_categorie)
utilisateur (1) —< evenement (id_salarie)
evenement (1) —< inscription >— (1) utilisateur
conteneur (1) —< box (id_conteneur)
annonce (1) —< order >— (1) utilisateur (acheteur)
utilisateur (1) —< document (id_user)
topic_forum (1) —< message_forum (id_topic)
utilisateur (1) —< notification
```

Comptes de test présents dans la base remplie

La base remplie contient les comptes de démonstration : `test.admin@renova.test`, `test.salarie@renova.test`, `test.pro@renova.test`, `test.client@renova.test` (+ variantes `@test.fr`). Mots de passe **hashés (bcrypt)** → non lisibles dans le dump. Voir `03_COMPTE_DE_DEMONSTRATION.md`.

12 — Docker, Nginx & déploiement

Rôle de Docker

Docker **empaquette chaque brique** du projet (base, API, front) dans un conteneur isolé et reproductible. Avantage : le projet tourne **à l'identique** sur n'importe quelle machine disposant de Docker, sans installer Go, MySQL ou un serveur web.

Rôle de Docker Compose

`docker-compose.yml` **orchestre** tous les conteneurs d'une seule commande : il définit les services, le réseau, les volumes (persistance) et l'ordre de démarrage.

- `docker-compose.yml` : stack **locale** (build des images depuis `API/` et `Frontend/`). Inclut aussi **phpMyAdmin** (interface web de la base, `http://localhost:8082`) pour le développement.
- `docker-compose-prod.yml` : stack **prod/VM** utilisant les images publiées sur Docker Hub (`ame1bdj/upcycle-backend`, `ame1bdj/upcycle-frontend`).

Services (local)

Service	Conteneur	Image / build	Port	Rôle
mysql	uc_mysql	mysql:8.0	interne	Base de données pa2026
backend	uc_backend	build ./API	8081	API Go
frontend	uc_frontend	build ./Frontend	WEB_PORT → 80	Nginx (front + proxy)
phpmyadmin	uc_phpmyadmin	phpmyadmin:latest	8082	Admin base (local)

Volumes (persistance)

- `mysql_data` → données MySQL.
- `uploads_data` → fichiers uploadés (images, PDF).
- `documents_data` → factures/contrats PDF.

Rôle de Nginx

Nginx (conteneur `frontend`, config `Frontend/nginx.conf`) : - **sert les fichiers statiques** du front (HTML/CSS/JS) ; - **fait office de reverse-proxy** : `/api/*` → API Go (`backend:8081`), `/uploads/*` → fichiers du back-end ; - **réécrit les URLs propres** (`/login`, `/admin`, `/annonces` ...) vers les fichiers `.html` correspondants.

En **production**, un second Nginx (sur la VM Ubuntu) ajoute le **HTTPS** et le domaine `upcycleconnect.pro` devant la stack Docker.

Lancer les conteneurs

```
# Local
WEB_PORT=8088 docker compose up -d --build

# Prod (sur la VM)
docker compose -f docker-compose-prod.yml pull
docker compose -f docker-compose-prod.yml up -d
```

Voir les logs

```
docker compose logs -f backend      # API en direct
docker compose logs -f frontend     # Nginx
```

```
docker compose logs --tail=100 mysql
docker ps # état + ports
```

Redémarrer un service

```
docker compose up -d --build backend # rebuild + redémarre l'API seule
docker compose up -d --build frontend # rebuild + redémarre le front seul
docker compose restart backend # redémarre sans rebuild
```

Prouver que le site est accessible depuis l'extérieur

- Ouvrir **https://upcycleconnect.pro/** depuis n'importe quel appareil (hors réseau local).
- Le certificat **HTTPS** est valide (cadenas dans le navigateur).
- L'en-tête serveur indique `nginx (Ubuntu)` sur une **IP publique** (VM), preuve que ce n'est pas du localhost.
- Vérification rapide : `curl -I https://upcycleconnect.pro/` → réponse `200 / HTTP 2`.

Schéma du flux

```
[Utilisateur Internet]
  | HTTPS (https://upcycleconnect.pro)
  v
[Nginx hôte (VM Ubuntu)] — TLS / domaine
  |
  v
[Conteneur frontend : Nginx]
  ├── fichiers statiques (Front)
  ├── /api/* → [Conteneur backend : API Go :8081]
  └── /uploads/* → fichiers (images, PDF)
                        |
                        v
                    [Conteneur mysql : base pa2026]
```

Mettre à jour la production

```
# 1) Construire et publier les images
docker build -t amelbdj/upcycle-backend:vNN ./API && docker push amelbdj/upcycle-backend:vNN
docker build -t amelbdj/upcycle-frontend:vNN ./Frontend && docker push amelbdj/upcycle-frontend:vNN
# 2) Sur la VM : mettre les tags à jour dans docker-compose-prod.yml, puis
docker compose pull && docker compose up -d
```

13 — Dépannage rapide (FAQ)

Réflexe n°1 : `docker ps` (tout tourne ?) puis `docker compose logs -f backend`.

Problème	Symptôme	Cause probable	Commande / fichier	Correction
Front ne démarre pas	uc_frontend s'arrête	Nginx ne trouve pas backend en upstream	<code>docker compose logs frontend ; Frontend/nginx.conf</code>	Lancer la stack complète (<code>docker compose up -d</code>), pas le front seul
API ne démarre pas	uc_backend redémarre en boucle	Erreur de compilation / MySQL pas prêt	<code>docker compose logs -f backend ; cd API && go build ./...</code>	Corriger l'erreur Go ; attendre <code>uc_mysql healthy</code>
Base ne répond pas	500 sur toutes les routes	MySQL non prêt / mauvais identifiants	<code>docker compose logs mysql ; API/bdd/db.go</code>	Attendre <code>healthy</code> ; vérifier <code>DB_HOST=mysql</code>
Erreur 401	Appels rejetés	Token absent/expiré	Console : <code>localStorage.getItem('token')</code> ;	Se reconnecter

Problème	Symptôme	Cause probable	Commande / fichier	Correction
			API/auth/jwt.go	
Erreur 403	« Accès refusé »	Mauvais rôle	localStorage.getItem('userRole') ; VerifyRoleMiddleware	Utiliser un compte du bon rôle
Erreur 500	Réponse 500	Erreur SQL / scan / nil	docker compose logs -f backend ; API/bdd/*Req.go	Lire la ligne d'erreur Go (colonne/table)
Erreur CORS	<i>blocked by CORS policy</i>	Handler sans OPTIONS /header Authorization	API/admin/<domaine>.go	Ajouter Access-Control-Allow-Headers : Content-Type, Authorization + route OPTIONS . N'arrive pas en prod (même origine via /api)
Docker ne build pas	docker build échoue	Erreur Go / fichier manquant	cd API && go build ./... ; API/Dockerfile	Corriger l'erreur affichée avant de rebuild
Variable d'env manquante	Stripe/DB par défaut	.env absent	docker exec uc_backend env grep DB_ ; .env	Copier .env.example → .env et remplir
Import SQL échoue / accents cassés	erreurs SQL ou « ? » à la place des accents	Import sans utf8mb4	—	Réimporter avec --default-character- set=utf8mb4
Endpoint introuvable (404)	404 sur une route	Route non déclarée	grep -r "chemin" API/route/ ; API/main.go	Vérifier HandleFunc + appel RoutesX() dans main.go
Site externe ne répond pas	upcycleconnect.pro inaccessible	VM/conteneur arrêté, Nginx hôte KO	Sur la VM : docker ps ; sudo systemctl status nginx	Relancer la stack (docker compose up -d) ; sudo systemctl restart nginx
Nginx ne redirige pas	404 sur /api/... ou URLs propres	Config Nginx non à jour	Frontend/nginx.conf ; sudo nginx -t	Vérifier les blocs location /api/ , /uploads/ , URLs propres ; recharger Nginx
413 Request Too Large (upload)	upload photo refusé	Limite Nginx hôte (1 Mo)	Nginx hôte de la VM	Ajouter client_max_body_size 30M; dans la conf Nginx hôte
Modif invisible	Rien ne change à l'écran	Cache navigateur / conteneur pas rebuild	—	Ctrl+Shift+R ; sinon docker compose up - d --build frontend

Méthode générale « ça ne marche pas » (4 étapes)

1. docker ps → les conteneurs sont-ils **Up** (mysql **healthy**) ?
2. docker compose logs -f backend → lire la vraie erreur.
3. Console navigateur (F12) → onglet **Réseau** : quel code HTTP ?
4. Reproduire l'appel en curl pour isoler front vs back.

14 — Captures d'écran à ajouter

Prendre ces captures sur <https://upcycleconnect.pro/> (ou en local) et les placer dans un sous-dossier DOSSIER_UTILISATION_LOGICIELS/captures/ .

#	Nom de fichier	Page / URL	Pourquoi	Section du dossier
1	accueil.png	/ (accueil)	Montrer le produit et l'entrée	01, 04
2	connexion.png	/login	Point d'entrée authentification	04
3	inscription.png	/register	Création de compte (rôles)	04
4	dashboard_particulier.png	/client	Espace particulier + stats	05
5	creation_annonce.png	/client (formulaire)	Fonction clé : créer une annonce	05
6	upload_photo.png	/client (choix fichier)	Upload d'image	05
7	marketplace.png	/annonces	Filtres/recherche annonces	05, 06

#	Nom de fichier	Page / URL	Pourquoi	Section du dossier
8	conteneurs_particulier.png	/client (section box)	Réseau de conteneurs	05
9	dashboard_pro.png	/pro	Espace professionnel (teal)	06
10	abonnement_pro.png	/pro (abonnement)	Freemium/Premium	06
11	projet_pro.png	/pro (projets)	Projets avant/après	06
12	dashboard_salarie.png	/salarie	Espace salarié	07
13	creation_evenement.png	/salarie/evenements	Création d'événement/formation	07
14	backoffice_admin.png	/admin	Vue d'ensemble back-office	08
15	validation_utilisateur.png	/admin/users	Valider/refuser un compte	08
16	validation_annonce.png	/admin/validations	Modération d'annonce	08
17	conteneurs.png	/admin/conteneurs	Gestion des casiers + rapport PDF	08
18	finances.png	/admin/finances	Transactions et commission	08
19	documents_pdf.png	/admin/documents	Factures/contrats PDF	08
20	notification_cloche.png	n'importe quelle page connectée	Notifications in-app	05, 08
21	api_postman.png	Postman / curl	Preuve d'appel API (token + réponse)	09
22	docker_compose.png	Terminal docker ps	Preuve des 3 conteneurs Up	12
23	site_externes_https.png	https://upcycleconnect.pro/ (barre d'URL + cadenas)	Preuve de déploiement externe	12
24	base_de_donnees.png	phpMyAdmin (localhost:8082) ou terminal SQL	Preuve de la base et des tables	11

Priorité (si peu de temps)

1. site_externes_https.png (preuve de mise en ligne).
2. backoffice_admin.png + validation_annonce.png (module fort).
3. dashboard_particulier.png + creation_annonce.png (parcours principal).
4. docker_compose.png (preuve de conteneurisation).
5. api_postman.png (preuve API).

15 — Checklist de rendu final (MyGES)

Livrables obligatoires

- [] **Code source complet** zippé (dossiers API/, Frontend/, db/, fichiers Docker).
- [] **Export SQL base vide** → db/export_db_vide.sql  (généré).
- [] **Export SQL base remplie** → db/export_db_remplie.sql  (généré).
- [] **Dossier d'utilisation** (ce dossier) en **PDF** → DOSSIER_COMPLET_UTILISATION_LOGICIELS.pdf .
- [] **URL publique** fonctionnelle → **https://upcycleconnect.pro/**.
- [] **Comptes de test** documentés (avec mots de passe **complétés**) → 03_COMPTE_DE_DEMONSTRATION.md .
- [] **README** à la racine du projet.
- [] **docker-compose.yml** (+ docker-compose-prod.yml).
- [] **.env.example** présent.
- [] **Documentation API** → 09_GUIDE_API_G0.md .
- [] **Preuve de lancement local** (capture docker ps / site sur localhost:8088).
- [] **Preuve de déploiement externe** (capture https://upcycleconnect.pro + cadenas HTTPS).

Vérifications techniques avant dépôt

- [] docker compose up -d --build fonctionne **sur une machine propre** (testé chez un camarade).
- [] La base s'importe automatiquement (ou via export_db_remplie.sql).

- [] Les 4 rôles se connectent et accèdent à leur espace.
- [] L'API répond (`/admin/login` → token ; route protégée → 401 sans token).
- [] Les uploads (photos d'annonces) fonctionnent en ligne (limite Nginx hôte à 30 Mo).
- [] Le site public est **accessible depuis l'extérieur** (réseau mobile, pas seulement le Wi-Fi local).

Points à finaliser (rappel du diagnostic)

- [] **Compléter les mots de passe** des comptes de test.
- [] **Ajouter les captures d'écran** (voir `14_CAPTURE_ECRANS_A_AJOUTER.md`).
- [] Mentionner honnêtement les éléments **absents/partiels** : Swagger (absent), tests automatisés (absents), vérification INSEE du SIRET (absente — seule la clé de Luhn est vérifiée), stockage `date_naissance` (absent), calcul du score par matériau (partiel).
- [] Externaliser les **secrets** (Stripe/OneSignal/JWT) via `.env` pour la version rendue.

Sécurité / cohérence

- [] Vérifier qu'aucun secret sensible réel ne traîne en clair dans le dépôt public.
- [] Vérifier que le fichier `.env` **réel** n'est pas commité (seul `.env.example`).

Conclusion

UpcycleConnect est une plateforme d'économie circulaire complète, déployée en production (<https://upcycleconnect.pro>) sur une infrastructure Docker + Nginx + HTTPS. Ce dossier couvre l'ensemble des produits logiciels : front-end web, API Go, base MySQL, conteneurisation et reverse-proxy. Les guides par rôle permettent une prise en main immédiate ; les parties techniques (installation, API, base, Docker) assurent la reproductibilité du projet. Les éléments absents ou partiels sont signalés honnêtement dans le diagnostic (section 0).